

## 6. Übungsblatt zur Vorlesung Computergraphik im WS 2025/26

Besprechung am Montag, Dienstag und Mittwoch, 15.12., 16.12 & 17.12.2025

### Aufgabe 1 *Rasterisierung* [10 Votierpunkte]

Diese Aufgabe behandelt die Objektraum-getriebene Rendering-Technik *Rasterisierung*. Bei dieser Technik werden aus graphischen Primitiven zusammengesetzte Objekte, unter Berücksichtigung der Verdeckungsverhältnisse, mittels Transformationen in das Bildschirmkoordinatensystem überführt und überlagert.

Als Grundlage ist das Programmskelett eines fast vollständig implementierten Rasterisierers gegeben. Das Programm verwendet für die Darstellung der Bild-Ergebnisse die, aus vorangegangenen Übungsblättern bekannte, *ImageViewer*-Klasse. Nach erfolgreicher Kompilierung und Ausführung sollte das Programm ein Fenster mit schwarzem Bild, sowie ein Menü anzeigen.

Das Programm ist in verschiedene Module aufgeteilt. Den Kern des Renderers bildet die Klasse *Rasterizer*. Desweiteren finden Sie im Ordner *Scene* verschiedene Objekte und Lichtquellen, die für die Generierung einer Szene verwendet werden können. Alle Objekte stellen ein Dreiecks-Mesh zur Verfügung.

Der fertig implementierte Rasterisierer soll drei Modi zur Visualisierung von graphischen Primitiven, in diesem Fall Dreiecken, beinhalten:

- **POINTS:** Es werden nur die Eckpunkte der Dreiecke gezeichnet.
- **WIREFRAME:** Es werden die Kanten der Dreiecke gezeichnet.
- **FILLED:** Dreiecke werden als ausgefüllte Formen gezeichnet.

Die Farben und Positionen innerhalb eines Objekts werden zwischen den Eckpunkten interpoliert: bei Visualisierung der Kanten linear, bei ausgefüllten Dreiecken baryzentrisch.

**Hinweis:** Um das Zeichnen zu beschleunigen, wird Parallelisierung mit Hilfe von *OpenMP* verwendet. Zum Debuggen des Programms empfiehlt es sich, die Parallelisierung zu deaktivieren. Kommentieren Sie hierzu an der zu untersuchenden Stelle die mit „`#pragma omp [...]`“ beginnenden Zeilen aus.

**Hinweis:** Verwenden Sie die automatische Update-Funktion am besten nur im Release-Modus, da das Zeichnen etwas Zeit in Anspruch nehmen kann.

1. **Modelltransformation** [1 Votierpunkt]

Implementieren Sie die Transformation der Punktpositionen und Normalen in der Funktion `Rasterizer::drawObject` in der Datei `Rasterizer.cpp`. An dieser Stelle werden die Koordinaten eines Punkts und die ihm zugeordnete Normale von Objektkoordinaten in Weltkoordinaten überführt. Verwenden Sie die Model-View-Matrix `global_trafo` und weisen Sie die neu berechnete Position und Normale in Weltkoordinaten den Variablen `point_world` und `normal_world` zu. Ist die Transformation korrekt implementiert, können bereits einige Elemente der Szene (ohne Berücksichtigung der Beleuchtung) gerendert werden.

2. **Beleuchtungsmodell** [1 Votierpunkt]

Implementieren Sie als richtungsabhängiges Beleuchtungsmodell, ebenfalls in der Funktion `Rasterizer::drawObject`, das Lambertsche Kosinusetz. Orientieren Sie sich dazu an der bereits vorhandenen Umsetzung der ambienten Beleuchtung. Die Farbe und Intensität der Lichtquelle werden dabei getrennt behandelt, d.h. die Intensität soll als multiplikativer Faktor auf die Farbkanäle der Lichtquelle angewendet werden. Der Alphakanal soll von der Intensität nicht beeinflusst werden.

Zudem soll die Intensität mit einem entfernungsabhängigen Faktor multipliziert werden, sodass weiter entfernte Lichtquellen das Objekt weniger stark beleuchten. Setzen Sie für diesen Faktor  $k_d$  die folgende Formel um:  $k_d = (0.001 + d^2)^{-1}$ . Dabei gibt  $d$  die Länge des Strahls von der Lichtquelle zur Oberfläche an.

3. **Interpolation** [1 Votierpunkt]

Um weiche Farbübergänge und kontinuierliche Beleuchtung zu erreichen, ist es nötig, Farben oder Vektoren innerhalb der Dreiecke zu interpolieren. Dies wird typischerweise durch die Multiplikation der zu interpolierenden Größe an den Eckpunkten des Dreiecks mit den baryzentrischen Koordinaten an der gesuchten Position umgesetzt. Implementieren Sie deshalb die Funktion `calculateBarycentricCoords` in der Datei `Math.cpp`. Anschließend sollte das Rendern der Dreiecks- und Würfel-Szene mit ausgefüllten Dreiecken möglich sein.

4. **Linienrasterisierung** [4 Votierpunkte]

Bisher können ausgefüllte Dreiecke oder deren Eckpunkte visualisiert werden. In dieser Teilaufgabe soll das Visualisieren der Gitterstruktur bzw. der Dreieckskanten umgesetzt werden. Diese Darstellung wird auch als *Wireframe* bezeichnet. Das wohl bekannteste Verfahren zum Rasterisieren von Linien ist der *Bresenham-Algorithmus*. Implementieren Sie diesen Algorithmus in der Funktion `Rasterizer::rasterizeLine` in der Datei `Rasterizer.cpp`. Die Funktion erhält die Endpunkte einer Linie in Bildschirmkoordinaten als Eingabeparameter. Die zur entsprechenden Linie gehörenden Pixel können mit der Funktion `Rasterizer::setPixel` gesetzt werden. Testen Sie Ihre Implementierung mit allen Szenen, da vor allem bei vertikalen Linien Artefakte auftreten können. Anschließend stehen alle Funktionen des Rasterisierers zur Verfügung.

5. **Theorie** [1 Votierpunkt]

Diese Teilaufgabe behandelt den Umgang mit Verdeckung und Objekten außerhalb des sichtbaren Bereichs. Bitte beantworten Sie die folgenden Fragen:

- (a) **Z-Buffer:** Der gegebene Rasterisierer verwendet einen sogenannten Z-Buffer. Wofür wird dieser gebraucht und welche Problematik ergibt sich beim parallelen Rendern, wie es z.B. auf der GPU geschieht?
- (b) **Clipping:** Für die verschiedenen Darstellungsmethoden des Rasterisierers werden unterschiedliche Ansätze für Clipping verwendet. Welche sind das und warum sind die Implementierungen im Falle der Wireframe-Methode und beim Darstellen der ausgefüllten Dreiecke eigentlich unzureichend?

#### 6. Eigene Szene [2 Votierpunkte]

Werden Sie kreativ und entwerfen Sie eine eigene Szene! Dazu können Sie die bereits vorhandenen Objekte aus dem *Scene* Ordner verwenden oder neue erstellen. Implementieren Sie hierzu die Funktion `createOwnScene` in der Datei *main.cpp*. Sie können sich dabei beispielsweise an `createComplexScene` orientieren.

Die Ergebnisse für die verschiedenen Szenen sollten wie folgt aussehen:

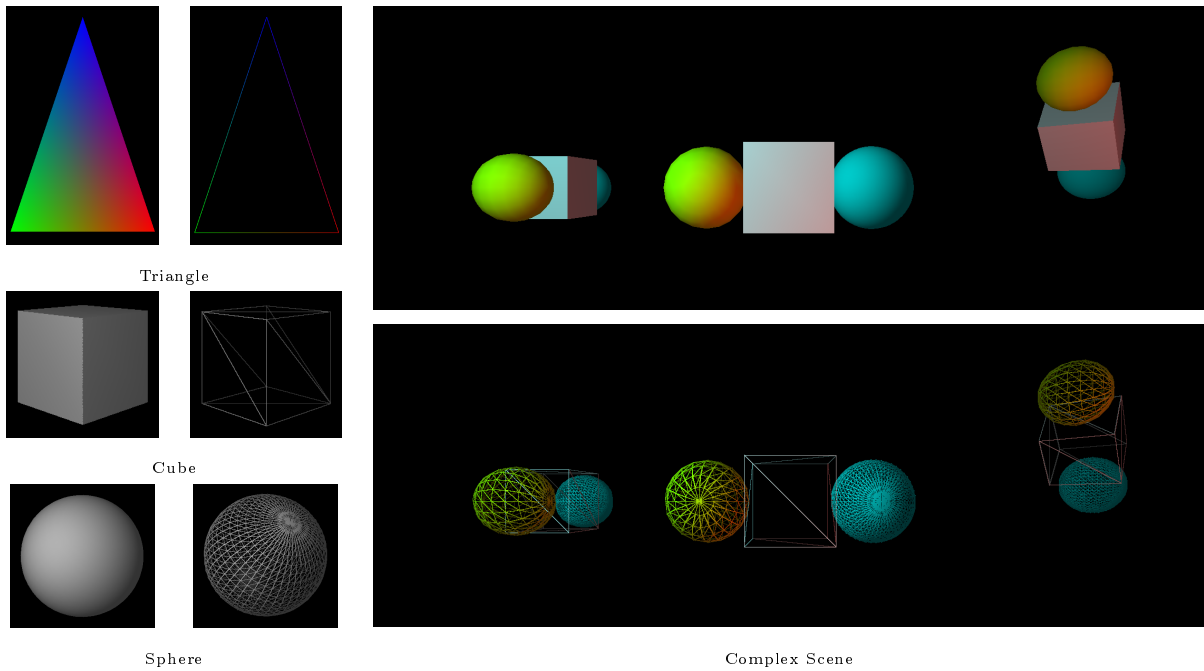


Abbildung 1: Darstellung der verschiedenen Szenen mit ambienter sowie Lambertscher Beleuchtung und Wireframe-Modus mit Linienrasterisierung.